# Authentication and Profile Remediation Report

Industry-grade engineering assessment and closure record

System: RGU NCC Cadet Management System

Release reference: c8ad8f1 / PR #26

Assessment date: 02 November 2026

RESOLVED

## Executive Summary

The system previously authenticated users successfully but could report that the associated profile could not be loaded. The principal defect was a concurrency race between the password login path and Supabase authentication state callbacks: a second profile request returned null immediately while the first request was still running. This converted a valid login into a false failure and could leave the interface in a misleading state.

The remediation is complete and merged. The implementation now coordinates concurrent profile requests, uses the authenticated access token for server-side profile reads, bounds all network-dependent profile operations, stabilizes the auth listener lifecycle, preserves legacy PIN migration, and prevents infrastructure/profile failures from consuming credential lockout attempts.

## Change Status Overview

| Area | Status | Engineering outcome |
|---|---|---|
| Concurrent profile loading | FIXED | In-flight profile requests are shared instead of returning false null failures. |
| Profile authorization | ADDED | Server action validates the access token and requires the token subject to match the requested profile ID. |
| Timeout and retry behavior | FIXED | Profile reads, session lookup, and profile creation are bounded and retried predictably. |
| Auth listener lifecycle | FIXED | Listener is installed once, disposed safely, and no longer depends on branding state changes. |
| Credential lockout | FIXED | Only credential failures increment the five-attempt, 60-second lockout; profile/server failures do not. |
| Legacy PIN migration | HARDENED | Successful legacy login remains usable even if password upgrade needs later retry. |
| Regression coverage | ADDED | Concurrent profile-fetch behavior is covered by an automated test. |

## Scope

# 1. Scope and System Context

This report covers the browser authority-card provider, the login and recovery interface, the server-side profile actions, the Supabase client boundary, and automated validation associated with the remediation. It does not certify external Supabase project configuration, production secrets, database migration execution, email delivery, or end-user device/network conditions.

Assessment boundary, evidence sources, and authentication architecture

## Evidence reviewed

**A.** Source implementation in src/lib/auth-context.tsx, src/app/page.tsx, and src/app/actions/profile-actions.ts.

**B.** Automated tests in tests/auth-context.test.tsx and the repository-wide Vitest suite.

**C.** Production build, TypeScript, and ESLint validation executed after the remediation.

**D.** Merged pull request #26 and commit c8ad8f1.

## Authentication flow

Users enter a role-specific identifier and PIN. The client derives the internal email candidates, attempts the salted password format, and then attempts the legacy PIN format where applicable. Once Supabase returns a user session, the provider loads the matching profile, maps it into the application User model, and redirects to the dashboard. Supabase auth state events independently restore sessions on reload and token refresh.

# 2. Findings and Resolution Register

| ID | Status | Finding | Resolution / evidence |
|---|---|---|---|
| F-01 | FIXED | Concurrent fetch race | A shared Promise prevents loginWithPassword and onAuthStateChange from treating the same in-flight request as a missing profile. |
| F-02 | FIXED | Unbounded retry path | Every server profile attempt is race-bounded by a timeout; retries are limited and include a controlled delay. |
| F-03 | ADDED | Server profile identity check | getProfileByIdAction verifies the bearer token and rejects a requested ID that does not match the authenticated subject. |
| F-04 | FIXED | Auth listener churn | The auth subscription no longer re-registers when unit branding state changes; disposal guards prevent late state writes. |
| F-05 | FIXED | False lockout classification | Profile loading/server failures are surfaced without incrementing failed-credential counters. |
| F-06 | HARDENED | Legacy password upgrade | A successful legacy login is not discarded if the post-login password upgrade fails; the failure is logged for follow-up. |
| F-07 | ADDED | Regression protection | A test verifies that a concurrent auth callback and login share one profile request and both complete successfully. |

## Profile acquisition control

The provider maintains a module-level Promise for the active profile load. Any overlapping caller awaits that Promise. This preserves a single source of truth for the resulting User object and eliminates the previous null-on-contention behavior. The reference is cleared only when the exact active Promise settles, preventing an older request from clearing a newer request.

# 3. Technical Remediation Detail

Implementation changes and operating behavior

## Session-bound server fallback

The browser client remains the first profile-read path when RLS permits self-read access. If that path does not return a row, the provider obtains the active access token and calls the server action. The server action validates the token with Supabase Auth and enforces subject-to-profile identity equality before querying profile data.

## Self-healing profile creation

If authentication is valid but no profile row exists, the provider attempts a server-side profile upsert using the active access token. This remains best-effort and time-bounded. A failed repair is logged explicitly and cannot silently become a success-shaped application user.

## Lifecycle and state safety

A. INITIAL_SESSION loads public unit branding only when no authenticated session is present.

B. SIGNED_IN and TOKEN_REFRESHED events do not cause redundant profile loads after a user is already established.

C. SIGNED_OUT clears user state and in-flight profile state, then returns to the login page.

D. Disposed listeners ignore late asynchronous responses, reducing stale state updates during navigation or remount.

## Credential lockout behavior

The login page continues to enforce five failed credential attempts followed by a 60-second client-side lockout. Empty input is rejected before authentication. A profile/server failure is displayed to the user but is not counted as a bad credential, preventing a healthy account from being locked because of a transient backend issue.

# 4. Validation and Acceptance Evidence

| Validation | Result | Evidence |
|---|---|---|
| Repository test suite (Vitest) | PASS | 8 test files; 28 tests passed. |
| TypeScript compiler | PASS | npx tsc --noEmit completed successfully. |
| ESLint | PASS | npm run lint -- --no-cache completed successfully. |
| Next.js production build | PASS | npm run build compiled, type-checked, and generated 15 routes. |
| Pull request integration | PASS | PR #26 merged into main. |
| Observed user outcome | PASS | User reported that login is now successful. |

## Acceptance criteria

PASS Valid credentials authenticate and profile loading completes without the previous false error.

PASS The 60-second lockout remains active for repeated credential failures.

PASS A slow or unavailable profile path cannot hold the login indefinitely.

PASS Profile/server errors do not consume credential-attempt budget.

PASS A legacy PIN user can enter the system even if password migration needs a later retry.

# 5. Residual Risk and Recommendations

## Residual risks

Controls that remain outside this code change

**R-01** The current lockout is client-side and browser-scoped. It is useful for user experience but is not a substitute for server-side rate limiting or Supabase Auth protections.

**R-02** The application uses a browser-managed Supabase session and passes access tokens to Server Actions. Production deployment must keep the service-role key server-only and must use HTTPS.

**R-03** Automatic profile creation assigns conservative defaults. Operational policy should define how new users are approved, assigned to a unit, and reviewed.

**R-04** The report validates application code and build behavior, not the live Supabase schema, RLS policy state, SMTP provider, DNS, or deployed environment variables.

## Recommended next controls

1. Enable Supabase/Auth provider rate limiting and monitor authentication failures centrally.
2. Add structured telemetry for profile-load failures with request correlation IDs, excluding tokens and personal secrets.
3. Add an end-to-end smoke test against a staging Supabase project covering salted login, legacy migration, session restore, logout, and missing-profile repair.
4. Verify migrations 005-007 and the production RLS policies are applied consistently before release promotion.
5. Rotate any credentials that may have appeared in historical setup documentation and keep production secrets in the deployment secret manager.

## Deployment checklist

- [ ] Confirm NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY are configured for the target environment.
- [ ] Confirm SUPABASE_SERVICE_ROLE_KEY exists only in server-side deployment configuration.
- [ ] Confirm the profiles table contains the required identity and unit columns, or that fallback behavior is intentionally accepted.
- [ ] Run the login smoke test with a real test account and verify dashboard redirection.
- [ ] Verify five bad attempts produce the 60-second lockout and that a profile outage does not increment the counter.

## 6. Closure Statement

The reported production symptom—successful authentication followed by a profile-fetch failure—has been addressed at its root in the client/server interaction model. The change is implemented, tested, built successfully, and merged through the protected-branch pull request workflow. The remaining recommendations are operational hardening items rather than blockers for the corrected login path.

| Artifact | Traceability |
| --- | --- |
| Authentication provider | src/lib/auth-context.tsx - profile coordination, session-bound fallback, timeout controls, listener lifecycle. |
| Login interface | src/app/page.tsx - input validation, legacy prefix fallback, lockout classification. |
| Profile server action | src/app/actions/profile-actions.ts - token validation and profile lookup fallback. |
| Regression test | tests/auth-context.test.tsx - concurrent profile fetch regression. |
| Commit | c8ad8f1 - fix: stabilize login and profile loading. |
| Pull request | CaptainCheetah978/RGU-NCC-Web-Application#26 - merged into main. |

## Status legend

FIXED Root cause corrected and validated.

ADDED **New control or coverage introduced.**

HARDENED **Existing behavior retained with improved failure handling.**

OPEN **Recommendation or environment-dependent item remains.**